Assignment 1 (Weeks 1 and 2)

PART 1

1.

s.no	Traditional	DBMS
1.	Data redundancy which wastes storage space.	Allows referencing between dataset, uses storage space efficiently.
2.	Data retrieval is cumbersome.	Data retrieval is easier and quicker.
3.	No consistency ensured across fields.	Data consistency is enforced.
4.	No way of controlling security on a fine grained basis.	Contains tools to control security on a fine grained basis.

Use case : For storing the data in the case of Hotels requires quick querying to reduce the waiting time for customer bookings, data consistency, quick updation as customers arrive daily in huge numbers, and security to protect the privacy of the customers. All these points would demand for a DBMS and a traditional file system would break down soon in such a scenario.

2.

Weak entity : double rectangle
Strong entity : rectangle

The primary key (Partial key) of a weak entity is formed using a relationship with the primary key of the strong entity it depends on. This relationship is called the identifying relationship.

3.

A multivalued attribute since it allows us to store multiple values for a single entity.

4.

A super key is any set of attributes that can uniquely identify a row.
E.g. (roll_no), (roll_no, name), (roll_no, email), etc.

A candidate key is the smallest possible super key. No attribute in the candidate key can be removed without losing the uniqueness of the entity.
E.g. {roll_no}, {email}

5.

Constraint violated : referential integrity (also called foreign key constraint)

It protects against having a standalone record in a table for which information cannot be obtained by referencing other tables.

Ideally the insert operation should throw an error.

6.

An additional table is created for representing the relationship where the primary key may be a composite key.

Its primary key consists of both the primary keys of the individual entity tables.

The relationship can not simply be represented as a column in one of the participating entity tables because that would create duplicate entries for the primary key field in that table.

PART 2

1.

Customer(cust_id, name, phone, address) , Primary key : cust_id
Restaurant(r_id, name, rating) , Primary key : r_id
Order(order_id, order_date) , Primary key : order_id
MenuItem(item_id, item_name, price) , Primary key : item_id

Associative entity : OrderItem(order_id, item_id, quantity) , Primary key : (order_id, item_ id)

Relationships

1. Customer — Places — Order

Cardinality = 1 : N

Justification : A customer can place multiple orders, but each order is placed by exactly one customer.

2. Restaurant — Receives/AssociatedWith — Order

Cardinality = 1 : N

Justification : A restaurant can receive many orders, but each order is associated with exactly one restaurant.

3. Restaurant — Offers — MenuItem

Cardinality = 1 : N

Justification : A restaurant can offer multiple menu items, and each menu item belongs to one restaurant.

4. Order — Contains — MenuItem

Cardinality = M : N

Justification : An order may contain many menu items, and the same menu item can appear in many different orders.

2.

One multivalued attribute : phone, a customer may have multiple phone numbers
One composite attribute : address, since it can be broken down into street, city, state, country

3.

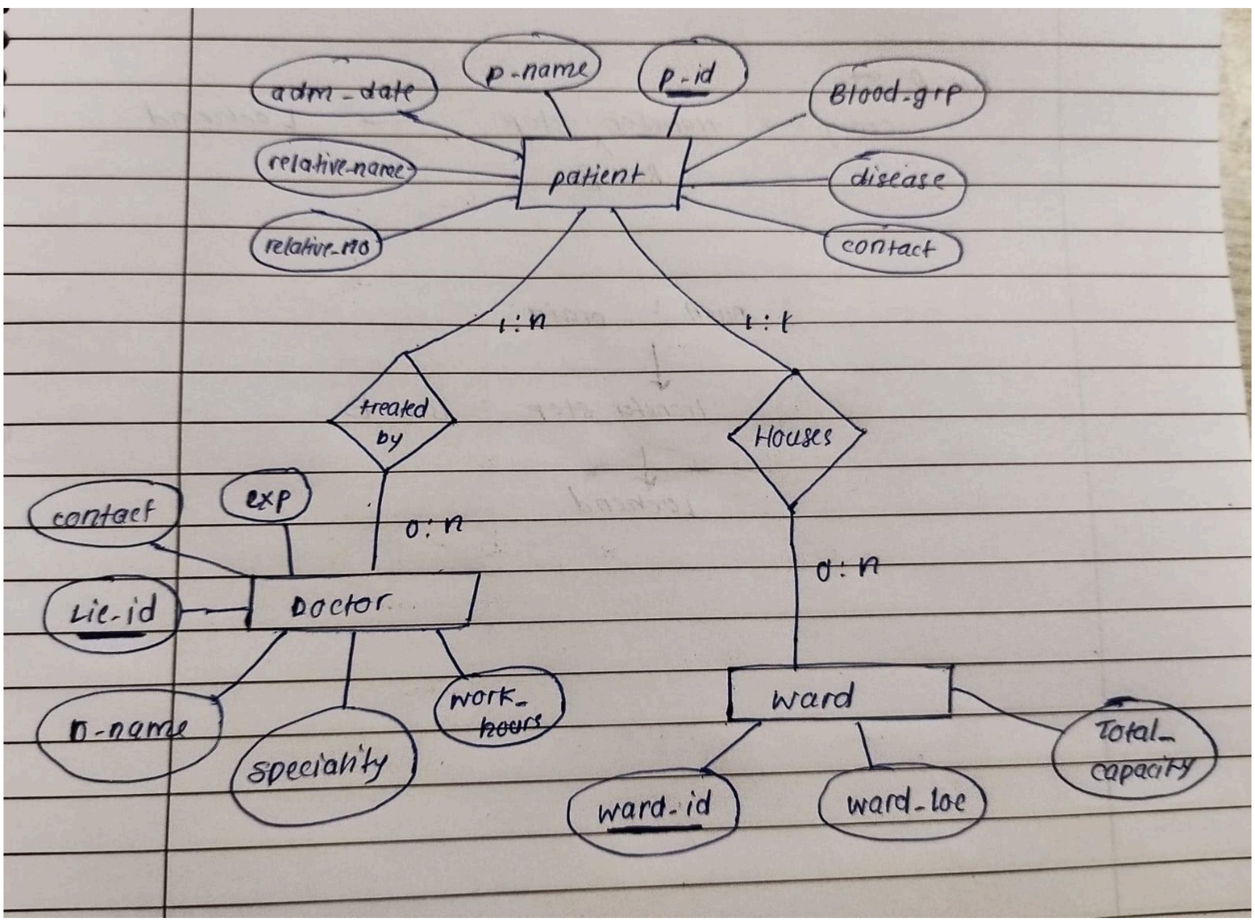
OrderItem(order_id, item_id, quantity)
Primary Key: (order_id, item_id)

Foreign Keys:
order_id → Order(order_id)
item_id → MenuItem(item_id)

Additional Attribute: quantity

PART 3

1.



2.

In the hospital database, the Doctor–Patient relationship is many-to-many (M:N) because a patient may be treated by multiple doctors during their stay, while a doctor can treat many different patients. This relationship creates an associative entity called Treatment. The composite primary key of Treatment would consist of (Lic_ID, P_ID), where Lic_ID is the doctor's primary key and P_ID is the patient's primary key.

For participation constraints, the Patient–Ward relationship has total participation on the Patient side because every admitted patient must be assigned to exactly one ward. The Ward side has partial participation since a ward may temporarily contain no patients. In the Doctor–Patient relationship, the patient has total participation because every patient in the system is assumed to receive treatment from at least one doctor. Doctor has partial participation because a doctor may exist in the database even when not currently treating any patients.

One design decision was modelling Relative_Name and Relative_Number as simple attributes of Patient rather than creating a separate entity. Since the scenario only requires storing basic emergency contact information, representing them as attributes keeps the ER model simpler while still capturing the required data.